LABORATORY PRACTICAL MANUAL

Machine Learning Practicals

Fourth Year of Computer Engineering

Savitribai Phule Pune University (SPPU)

Name: _____

Roll No.: _____ Division: _____

Academic Year: _____

Journal Signature: _____

INDEX

Exp. No.	Title of Experiment	Date	Sign
01	Predict the Price of an Uber Ride using Linear Regression and Random Forest Regression		
02	Email Spam Detection using K-Nearest Neighbors and Support Vector Machine		
03	Bank Customer Churn Prediction using a Neural Network Classifier		
04	Gradient Descent Algorithm to Find the Local Minima of a Function		
05	K-Nearest Neighbors Algorithm on the Diabetes Dataset		

EXPERIMENT NO: 01

Title:

Predict the Price of an Uber Ride using Linear Regression and Random Forest Regression

Aim:

To predict the fare of an Uber ride from a given pickup point to the agreed drop-off location by pre-processing the dataset, identifying outliers, checking correlation, implementing Linear Regression and Random Forest Regression models, and comparing their performance using R2 score, RMSE and MAE.

Objectives:

- To understand the concept of regression and its use in price prediction.
- To pre-process a real-world dataset (missing values, invalid records, date-time features).
- To calculate the trip distance from latitude and longitude using the Haversine formula.
- To identify outliers using box plots and the IQR method.
- To study the correlation between the input features and the fare amount.
- To implement Linear Regression and Random Forest Regression models.
- To evaluate and compare the models using R2, RMSE and MAE.

Hardware Requirements:

- Laptop/Desktop Computer
- Intel Core i3 Processor or above (i5 recommended)
- Minimum 4 GB RAM (8 GB recommended)
- Keyboard and Mouse
- Internet Connection (to download the dataset and libraries)

Software Requirements:

- Operating System: Linux
- Programming Language: Python 3.8 or above
- IDE: Jupyter Notebook / Google Colab / VS Code
- Web Browser: Google Chrome, Mozilla Firefox
- Python Libraries: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn
- Dataset: uber.csv (Uber Fares Dataset, Kaggle - yasserh/uber-fares-dataset)

Theory Concepts:

Regression

Regression is a supervised learning technique used to predict a continuous numeric value (such as fare, price or temperature) from one or more input features. The model learns the relationship between the independent variables (X) and the dependent variable (y) from labelled training data.

Linear Regression

Linear Regression assumes a straight-line relationship between the inputs and the output. For multiple features the model is:

$$y = b0 + b1x1 + b2x2 + ... + bnxn$$

The coefficients b0...bn are found by the Ordinary Least Squares (OLS) method, which minimises the sum of squared differences between actual and predicted values. It is fast and easy to interpret, but it cannot capture non-linear relationships.

Random Forest Regression

Random Forest is an ensemble method that builds many decision trees on random subsets of the data and features (bagging). For regression, the final prediction is the average of the predictions of all trees. Averaging reduces variance and overfitting, and the model can learn non-linear relationships and feature interactions. It also provides feature importance scores.

Data Pre-processing

Raw data must be cleaned before modelling. In this experiment pre-processing includes: removing identifier columns (Unnamed: 0, key), dropping rows with missing values, converting pickup_datetime to a date-time type and extracting year, month, day, day of week and hour, removing invalid records (fare <= 0, impossible coordinates, passenger count of 0 or more than 6), and creating a new feature distance_km.

Haversine formula (great-circle distance between two points on Earth):

$$a = sin2(dlat/2) + cos(lat1) \times cos(lat2) \times sin2(dlon/2)$$
$$d = 2R \times arcsin(sqrt(a)), \quad R = 6371 \text{ km}$$

Outliers

An outlier is a value that differs greatly from the other observations (for example a fare of 500 dollars for a 1 km trip). Outliers distort the regression line and inflate the error. They are detected with a box plot and the Inter-Quartile Range (IQR) method:

$$IQR = Q3 - Q1$$
$$\text{Lower bound} = Q1 - 1.5 \times IQR, \quad \text{Upper bound} = Q3 + 1.5 \times IQR$$

Values outside these bounds are treated as outliers and removed.

Correlation

Correlation measures the strength and direction of the linear relationship between two variables. The Pearson correlation coefficient r lies between -1 and +1: +1 is a perfect positive relationship, -1 a perfect negative relationship and 0 means no linear relationship. A heat map of the correlation matrix shows which features (such as distance_km) are strongly related to fare_amount.

Evaluation Metrics

R2 Score = 1 - (SS_res / SS_tot) - proportion of variance explained by the model (closer to 1 is better).

RMSE = sqrt((1/n) x sum (y - y_pred)^2) - average error in the units of fare; penalises large errors.

MAE = (1/n) x sum |y - y_pred| - average absolute error.

Dataset Description

The Uber Fares dataset contains about 200,000 rides with the columns: fare_amount (target), pickup_datetime, pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude and passenger_count (plus the index columns Unnamed: 0 and key).

Algorithm / Steps:

1. Import the required libraries and load uber.csv.
2. Inspect the data (head, info, missing values).
3. Drop unwanted columns and rows with null values.
4. Convert pickup_datetime and extract year, month, day, day of week and hour.
5. Compute distance_km with the Haversine formula and remove invalid records.
6. Draw box plots and remove outliers using the IQR method.
7. Plot the correlation heat map and study the correlation with fare_amount.
8. Split the data into training (80%) and testing (20%) sets.
9. Train the Linear Regression and Random Forest Regression models.
10. Predict on the test set and calculate R2, RMSE and MAE.
11. Compare the models and conclude which one performs better.

Program:

The complete program is provided in the file uber_fare_prediction.py in the folder of this practical (see the accompanying code package).

Observation / Output:

Model	R2 Score	RMSE	MAE
Linear Regression			
Random Forest Regression			

Run the program on the Kaggle dataset, paste the screenshots of the box plots, correlation heat map and the results table here, and fill the observation table above. Generally the Random Forest model gives a higher R2 and a lower RMSE than Linear Regression because the fare has a non-linear relationship with distance, time and location.

Outcomes:

- Understand regression and the working of Linear and Random Forest regression.
- Perform data cleaning and feature engineering on a real dataset.
- Detect and remove outliers using the IQR method.
- Interpret a correlation matrix.
- Evaluate regression models using R2, RMSE and MAE.
- Compare two models and select the better one.

Conclusion:

In this practical the Uber fares dataset was pre-processed, the trip distance was derived using the Haversine formula, outliers were removed with the IQR method and the correlation of the features with the fare was analysed. Linear Regression and Random Forest Regression models were implemented and compared using R2, RMSE and MAE. The distance of the trip was found to be the most influential feature. The model with the higher R2 and lower RMSE (generally Random Forest) is the better predictor of the ride fare.

Viva Questions:

1. What is the difference between regression and classification?
2. Why is the Haversine formula used instead of Euclidean distance?
3. How does the IQR method detect outliers?
4. Why does Random Forest usually outperform Linear Regression on this dataset?
5. What do R2 and RMSE indicate?

EXPERIMENT NO: 02

Title:

Email Spam Detection using K-Nearest Neighbors and Support Vector Machine

Aim:

To classify emails as Spam or Not Spam using the binary classification method with the K-Nearest Neighbors (KNN) and Support Vector Machine (SVM) algorithms, and to analyse and compare their performance.

Objectives:

- To understand binary classification and its use in spam detection.
- To understand the normal state (Not Spam) and the abnormal state (Spam).
- To implement the KNN and SVM classifiers.
- To split the data into training and testing sets.
- To evaluate the models using the confusion matrix, accuracy, precision, recall and F1-score.
- To study the effect of K on the accuracy of KNN.
- To compare KNN and SVM and decide which is better for this problem.

Hardware Requirements:

- Laptop/Desktop Computer
- Intel Core i3 Processor or above (i5 recommended)
- Minimum 4 GB RAM (8 GB recommended)
- Keyboard and Mouse
- Internet Connection (to download the dataset and libraries)

Software Requirements:

- Operating System: Linux
- Programming Language: Python 3.8 or above
- IDE: Jupyter Notebook / Google Colab / VS Code
- Web Browser: Google Chrome, Mozilla Firefox
- Python Libraries: Pandas, Matplotlib, Scikit-learn
- Dataset: emails.csv (Email Spam Classification Dataset, Kaggle - balaka18)

Theory Concepts:

Binary Classification

Binary classification is a supervised learning task where every sample is assigned to one of two classes. In spam detection the two states are: (a) Normal State - Not Spam (class 0) and (b) Abnormal State - Spam (class 1). The model learns from labelled emails and predicts the class of unseen emails.

K-Nearest Neighbors (KNN)

KNN is a simple, non-parametric, instance-based (lazy) algorithm. It stores all training samples and, to classify a new sample, finds the K closest training samples using a distance measure and assigns the majority class among them.

$$Euclidean\ distance:\ d(p, q) = sqrt(\ sum\ (pi - qi)^2\ )$$

A small K makes the model sensitive to noise (overfitting) while a large K makes it too smooth (underfitting). An odd value of K avoids ties in binary classification. KNN becomes slow for large, high-dimensional data such as the 3000 word-count features of this dataset.

Support Vector Machine (SVM)

SVM finds the hyperplane that separates the two classes with the maximum margin. The training points that lie closest to the hyperplane are called support vectors and they alone define the decision boundary. The hyperplane is $w.x + b = 0$ and the margin is $2/||w||$.

The parameter C controls the trade-off between a wide margin and misclassification of training points. Kernels (linear, polynomial, RBF) allow SVM to handle non-linear data. A linear kernel works very well for high-dimensional text data.

Confusion Matrix and Performance Measures

For the Spam class: TP = spam correctly detected, TN = not-spam correctly detected, FP = not-spam wrongly marked as spam, FN = spam missed.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1\text{-}Score = 2 \times Precision \times Recall / (Precision + Recall)$$

In spam filtering a high precision is important because an important mail should not be moved to the spam folder (FP), and a high recall is needed so that spam does not reach the inbox (FN).

Dataset Description

emails.csv has 5172 rows and 3002 columns. The first column Email No. is only an identifier, the next 3000 columns are the counts of the 3000 most common words in the emails, and the last column Prediction is the label (1 = Spam, 0 = Not Spam).

Algorithm / Steps:

1. Import the libraries and load emails.csv.
2. Check the shape, class distribution and missing values.
3. Drop the Email No. column and separate the features (X) and the target (Prediction).
4. Split the data into training (80%) and testing (20%) sets using stratified sampling.
5. Train the KNN classifier with K = 5.
6. Train the SVM classifier with a linear kernel.
7. Predict the test data with both models.
8. Compute the confusion matrix, accuracy, precision, recall and F1-score.
9. Plot the confusion matrices and the graph of accuracy versus K.

10. Compare the two models and write the conclusion.

Program:

The complete program is provided in the file email_spam_detection.py in the folder of this practical (see the accompanying code package).

Observation / Output:

Model	Accuracy	Precision	Recall	F1-Score
KNN (K = 5)				
SVM (linear kernel)				

Run the program on emails.csv, paste the screenshots of the confusion matrices and the accuracy versus K graph here, and fill the observation table above. On this dataset SVM generally gives higher accuracy and recall than KNN, and it predicts faster because only the support vectors are used.

Outcomes:

- Understand binary classification and the two states of spam detection.
- Implement KNN and SVM classifiers using Scikit-learn.
- Read and interpret a confusion matrix.
- Calculate accuracy, precision, recall and F1-score.
- Analyse the effect of K on KNN.
- Compare two classifiers on the same dataset.

Conclusion:

In this practical emails were classified as Spam or Not Spam using the KNN and SVM algorithms. The performance of both classifiers was analysed using the confusion matrix, accuracy, precision, recall and F1-score. SVM, which separates the classes using a maximum-margin hyperplane, handles the high-dimensional word-count data better than KNN, which depends on distances in a 3000-dimensional space. Hence SVM is the more suitable classifier for email spam detection in this experiment.

Viva Questions:

1. What are the normal and abnormal states in spam detection?
2. How does the value of K affect KNN?
3. What are support vectors and what is the margin?
4. Why is precision important in spam filtering?
5. Why does KNN become slow on this dataset?

EXPERIMENT NO: 03

Title:

Bank Customer Churn Prediction using a Neural Network Classifier

Aim:

To build a neural network based classifier that determines whether a bank customer will leave the bank in the next 6 months, and to evaluate it using the accuracy score and the confusion matrix.

Objectives:

- To understand the architecture and working of an Artificial Neural Network (ANN).
- To read the dataset and separate the feature set and the target set.
- To encode categorical data and normalise the train and test data.
- To build, compile and train a neural network using Keras/TensorFlow.
- To identify points of improvement and implement them (Dropout, Early Stopping, class weights).
- To print the accuracy score and the confusion matrix.

Hardware Requirements:

- Laptop/Desktop Computer
- Intel Core i3 Processor or above (i5 recommended)
- Minimum 4 GB RAM (8 GB recommended)
- Keyboard and Mouse
- Internet Connection (to download the dataset and libraries)

Software Requirements:

- Operating System: Linux
- Programming Language: Python 3.8 or above
- IDE: Jupyter Notebook / Google Colab / VS Code
- Web Browser: Google Chrome, Mozilla Firefox
- Python Libraries: NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow (Keras)
- Dataset: Churn_Modelling.csv (Bank Customer Churn Modeling, Kaggle - barelydedicated)

Theory Concepts:

Customer Churn

Churn means that a customer stops using the services of a company. Predicting churn helps a bank to identify customers at risk and take retention steps. It is a binary classification problem: Exited = 1 (customer leaves) and Exited = 0 (customer stays).

Artificial Neural Network

An ANN is made of layers of interconnected neurons: an input layer (one node per feature), one or more hidden layers and an output layer. Each neuron computes a weighted sum of its inputs plus a bias and passes it through an activation function:

$$z = w1x1 + w2x2 + ... + wnxn + b, \quad a = f(z)$$

Forward propagation computes the output; the loss is measured; back-propagation computes the gradients and an optimiser (Adam) updates the weights to reduce the loss.

Activation Functions and Loss

ReLU: f(z) = max(0, z) - used in hidden layers, fast and avoids vanishing gradients.

Sigmoid: f(z) = 1 / (1 + e^(-z)) - used in the output layer of a binary classifier, gives a probability between 0 and 1 (probability > 0.5 means the customer leaves).

Binary cross-entropy loss: L = -[y log(p) + (1 - y) log(1 - p)].

Pre-processing and Normalisation

Identifier columns (RowNumber, CustomerId, Surname) carry no predictive information and are dropped. Categorical columns (Geography, Gender) are converted to numbers using one-hot encoding. Features such as Balance and EstimatedSalary are on very different scales from Age or Tenure, so the data is standardised: z = (x - mean) / standard deviation. The scaler is fitted on the training data only and then applied to the test data to avoid data leakage.

Points of Improvement

Overfitting - the model performs well on training data but poorly on new data. It is reduced using Dropout (randomly switching off neurons during training) and Early Stopping (stop training when the validation loss stops improving).

Class imbalance - only about 20% of customers leave, so the model is biased towards the majority class. Class weights give more importance to the minority class and increase recall for churners.

Other improvements: more neurons or layers, tuning the learning rate, batch size and epochs, and adjusting the decision threshold.

Dataset Description

The dataset has 10,000 records and 14 columns: RowNumber, CustomerId, Surname, CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary and Exited (target).

Algorithm / Steps:

1. Import the libraries and read Churn_Modelling.csv.
2. Drop RowNumber, CustomerId and Surname.
3. One-hot encode Geography and Gender; separate features X and target y (Exited).
4. Divide the data into training (80%) and test (20%) sets.
5. Normalise the data using StandardScaler (fit on train, transform train and test).
6. Build and train a base model (two hidden layers of 6 neurons) and note its accuracy.

7. Build the improved model with more neurons, Dropout layers, Early Stopping and class weights.
8. Predict on the test set (threshold 0.5).
9. Print the accuracy score, confusion matrix and classification report.
10. Plot the training and validation loss and write the conclusion.

Program:

The complete program is provided in the file churn_prediction_nn.py in the folder of this practical (see the accompanying code package).

Observation / Output:

Model	Accuracy	Recall (Churn class)
Base model		
Improved model		

Run the program on Churn_Modelling.csv, paste the screenshot of the accuracy score, confusion matrix and loss curves here, and fill the observation table above. Note that class weights usually lower the overall accuracy slightly but raise the recall of customers who leave, which is the more useful result for a bank.

Outcomes:

- Understand the structure and training of a neural network.
- Pre-process mixed numeric and categorical data.
- Build a neural network classifier using Keras.
- Apply Dropout, Early Stopping and class weights to improve a model.
- Interpret the accuracy score and confusion matrix.

Conclusion:

In this practical a neural network classifier was built to predict whether a bank customer will leave within the next 6 months. The data was pre-processed, normalised and divided into training and test sets. A base model was improved using Dropout, Early Stopping and class weights, and the final model was evaluated using the accuracy score and the confusion matrix. The experiment shows that accuracy alone is not enough for an imbalanced problem such as churn, and recall for the churn class should also be examined.

Viva Questions:

1. What is the role of an activation function?
2. Why is the sigmoid function used in the output layer?
3. Why must the scaler be fitted only on the training data?
4. How do Dropout and Early Stopping reduce overfitting?
5. Why is accuracy misleading on an imbalanced dataset?

EXPERIMENT NO: 04

Title:

Gradient Descent Algorithm to Find the Local Minima of a Function

Aim:

To implement the Gradient Descent algorithm to find the local minimum of the function y = (x + 3)^2 starting from the point x = 2.

Objectives:

- To understand the concept of optimisation and the gradient of a function.
- To understand the working of the Gradient Descent algorithm.
- To study the effect of the learning rate on convergence.
- To implement Gradient Descent in Python.
- To visualise the steps taken towards the minimum.

Hardware Requirements:

- Laptop/Desktop Computer
- Intel Core i3 Processor or above (i5 recommended)
- Minimum 4 GB RAM (8 GB recommended)
- Keyboard and Mouse
- Internet Connection (to download the dataset and libraries)

Software Requirements:

- Operating System: Linux
- Programming Language: Python 3.8 or above
- IDE: Jupyter Notebook / Google Colab / VS Code
- Web Browser: Google Chrome, Mozilla Firefox
- Python Libraries: NumPy, Matplotlib

Theory Concepts:

Optimisation

Optimisation means finding the input that gives the minimum (or maximum) value of a function. In machine learning, training a model means minimising a loss function with respect to the model parameters.

Gradient Descent

Gradient Descent is an iterative first-order optimisation algorithm. The gradient (derivative) of a function gives the direction of steepest increase, so moving in the opposite direction decreases the function. Starting from an initial point, the algorithm repeatedly updates:

$$x_new = x_old - alpha \times f'(x_old)$$

where alpha is the learning rate (step size) and f'(x) is the gradient. The process stops when the change in x becomes smaller than a tolerance or when the maximum number of iterations is reached.

Learning Rate

If alpha is too small, convergence is very slow. If alpha is too large, the algorithm overshoots and may oscillate or diverge. For the function of this experiment, the update factor is (1 - 2 x alpha): with alpha = 0.1 the factor is 0.8 and the algorithm converges smoothly; with alpha = 1 it oscillates between two points; with alpha > 1 it diverges.

Local and Global Minima

A local minimum is a point lower than all its neighbouring points, whereas the global minimum is the lowest point of the whole function. For a convex function such as y = (x + 3)^2 there is only one minimum, so the local minimum is also the global minimum. For non-convex functions Gradient Descent may get stuck in a local minimum.

Problem Solution

Function: y = f(x) = (x + 3)^2

Derivative: f'(x) = 2(x + 3)

Starting point: x = 2, learning rate alpha = 0.1

Iteration 1: x = 2 - 0.1 x 2(2 + 3) = 2 - 1.0 = 1.0

Iteration 2: x = 1 - 0.1 x 2(1 + 3) = 1 - 0.8 = 0.2

Iteration 3: x = 0.2 - 0.1 x 2(0.2 + 3) = 0.2 - 0.64 = -0.44

Continuing in this way x approaches -3, where f'(x) = 0 and y = 0. The exact minimum is at x = -3.

Algorithm / Steps:

1. Define the function f(x) = (x + 3)^2 and its derivative f'(x) = 2(x + 3).
2. Initialise x = 2, learning rate = 0.1, tolerance = 1e-6 and maximum iterations = 100.
3. Calculate the gradient at the current x.
4. Update x = x - learning_rate x gradient.
5. Store the new value of x for plotting.
6. Repeat steps 3-5 until |x_new - x_old| < tolerance or the maximum iterations are reached.
7. Print the final x and y, and plot the function with the steps taken.

Program:

The complete program is provided in the file gradient_descent.py in the folder of this practical (see the accompanying code package).

Observation / Output:

```
Iter |      x      |    f(x)    | gradient
   1 |    1.000000 |  16.000000 | 10.000000
   2 |    0.200000 |  10.240000 |  8.000000
   3 |   -0.440000 |   6.553600 |  6.400000
   4 |   -0.952000 |   4.194304 |  5.120000
   5 |   -1.361600 |   2.684355 |  4.096000
  10 |   -2.463129 |   0.288230 |  1.342177
  20 |   -2.942354 |   0.003323 |  0.144115
  30 |   -2.993810 |   0.000038 |  0.015474
  40 |   -2.999335 |   0.000000 |  0.001662
```

```
  50 |   -2.999929 |   0.000000 |  0.000178
  60 |   -2.999992 |   0.000000 |  0.000019
Converged after 63 iterations
Local minimum at x = -2.999996, y = 0.0000000000
```

The algorithm converged in 63 iterations to x = -3.0 (approximately), where y = 0, which matches the exact minimum found analytically from f'(x) = 2(x + 3) = 0.

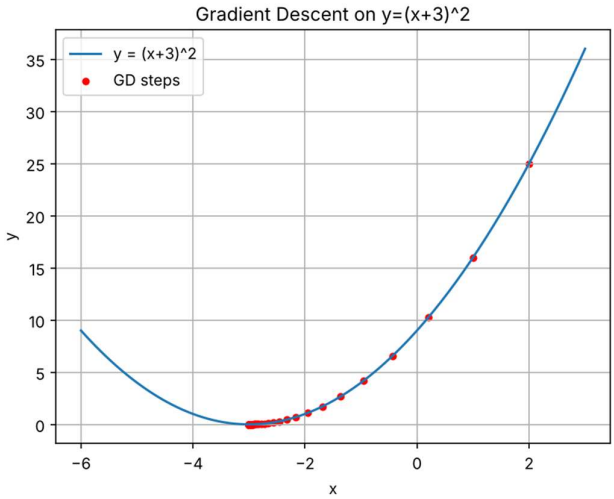


Figure: Gradient descent steps on y = (x + 3)^2 starting from x = 2

Outcomes:

- Understand the idea of the gradient and the update rule of Gradient Descent.
- Implement Gradient Descent from scratch in Python.
- Understand the effect of the learning rate on convergence.
- Distinguish between local and global minima.
- Visualise the optimisation path of an algorithm.

Conclusion:

In this practical the Gradient Descent algorithm was implemented to find the local minimum of the function y = (x + 3)^2 starting from x = 2 with a learning rate of 0.1. The value of x moved step by step towards -3 and the algorithm converged after 63 iterations, giving x = -3 and y = 0. The practical showed that the learning rate controls the speed and stability of convergence, and that Gradient Descent is the basic technique used for training machine learning models.

Viva Questions:

1. What is the gradient of a function?
2. Why do we move in the direction opposite to the gradient?
3. What happens if the learning rate is too large or too small?
4. What is the difference between a local and a global minimum?
5. Where is Gradient Descent used in machine learning?

EXPERIMENT NO: 05

Title:

K-Nearest Neighbors Algorithm on the Diabetes Dataset

Aim:

To implement the K-Nearest Neighbors algorithm on the diabetes.csv dataset and to compute the confusion matrix, accuracy, error rate, precision and recall.

Objectives:

- To understand the working of the K-Nearest Neighbors algorithm.
- To pre-process medical data containing invalid zero values.
- To scale the features for a distance-based algorithm.
- To implement KNN using Scikit-learn.
- To compute the confusion matrix, accuracy, error rate, precision and recall.
- To find the best value of K.

Hardware Requirements:

- Laptop/Desktop Computer
- Intel Core i3 Processor or above (i5 recommended)
- Minimum 4 GB RAM (8 GB recommended)
- Keyboard and Mouse
- Internet Connection (to download the dataset and libraries)

Software Requirements:

- Operating System: Linux
- Programming Language: Python 3.8 or above
- IDE: Jupyter Notebook / Google Colab / VS Code
- Web Browser: Google Chrome, Mozilla Firefox
- Python Libraries: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn
- Dataset: diabetes.csv (Kaggle - abdallamahgoub/diabetes)

Theory Concepts:

K-Nearest Neighbors

KNN is a supervised, instance-based algorithm used for classification and regression. It does not build a model during training; it stores the training data and makes the prediction only when a new sample arrives (lazy learning). For a new sample, KNN (1) calculates its distance from every training sample, (2) selects the K nearest samples and (3) assigns the class that occurs most often among them.

Distance Measures

Euclidean distance: d = sqrt(sum (xi - yi)^2)

Manhattan distance: d = sum |xi - yi|

$$Minkowski\ distance:\ d = (sum\ |xi - yi|^p)^(1/p)$$

Choosing K and Feature Scaling

A small K gives a flexible boundary that may overfit the noise, while a large K gives a smoother boundary that may underfit. The best K is selected by testing different values and comparing the accuracy. Because KNN is based on distance, features with large ranges (such as Insulin) would dominate features with small ranges (such as DiabetesPedigreeFunction); therefore the features are standardised before applying KNN.

Confusion Matrix and Measures

For the Diabetes (positive) class: TP = diabetic patients predicted diabetic, TN = non-diabetic predicted non-diabetic, FP = non-diabetic predicted diabetic, FN = diabetic predicted non-diabetic.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Error\ Rate = (FP + FN) / (TP + TN + FP + FN) = 1 - Accuracy$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

In medical diagnosis a high recall is very important because a missed patient (FN) is more harmful than a false alarm (FP).

Dataset Description

The diabetes dataset (Pima Indians Diabetes) has 768 records, 8 input features - Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction and Age - and the target Outcome (1 = diabetic, 0 = not diabetic). The columns Glucose, BloodPressure, SkinThickness, Insulin and BMI contain zeros that are physically impossible and represent missing values.

Algorithm / Steps:

1. Import the libraries and load diabetes.csv.
2. Replace the invalid zeros in Glucose, BloodPressure, SkinThickness, Insulin and BMI with NaN and fill them with the column median.
3. Separate the features (X) and the target (Outcome).
4. Split the data into training (80%) and testing (20%) sets.
5. Standardise the features using StandardScaler.
6. Train the KNN classifier with K = 5 and Euclidean distance.
7. Predict the test data.
8. Compute the confusion matrix, accuracy, error rate, precision and recall.
9. Plot the accuracy for K = 1 to 20 and choose the best K.

Program:

The complete program is provided in the file knn_diabetes.py in the folder of this practical (see the accompanying code package).

Observation / Output:

Measure	Value
True Positive (TP)	33
True Negative (TN)	83
False Positive (FP)	17
False Negative (FN)	21
Accuracy	0.7532 (75.32%)
Error Rate	0.2468 (24.68%)
Precision	0.6600
Recall	0.6111
Best K	8

Confusion Matrix (K = 5, 154 test records): [[83 17] [21 33]]

The Pima diabetes dataset (768 records) was split 80:20, giving 154 test records. With K = 5 the model correctly classified 116 of 154 records (accuracy 75.32%, error rate 24.68%). Of the 54 diabetic patients in the test set, 33 were detected and 21 were missed (recall 61.11%), and of the 50 patients predicted diabetic, 33 were truly diabetic (precision 66.00%). The accuracy-versus-K graph shows the best accuracy at K = 8.

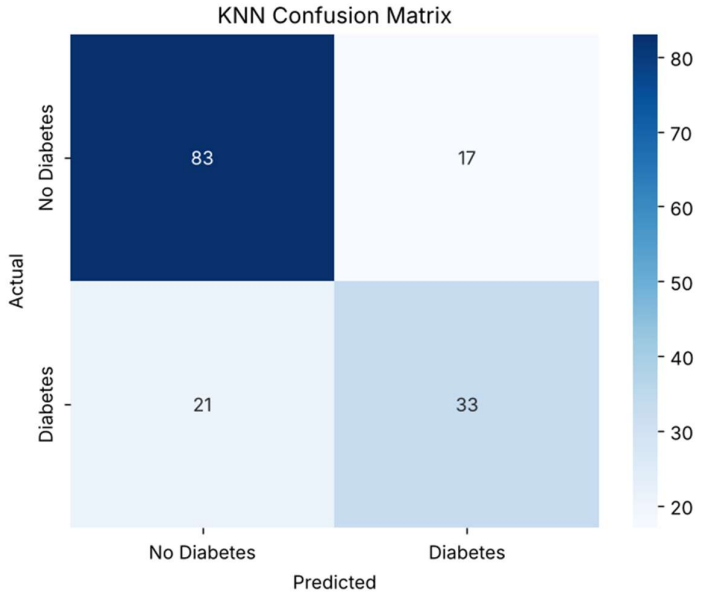


Figure: Confusion matrix of the KNN model (K = 5)

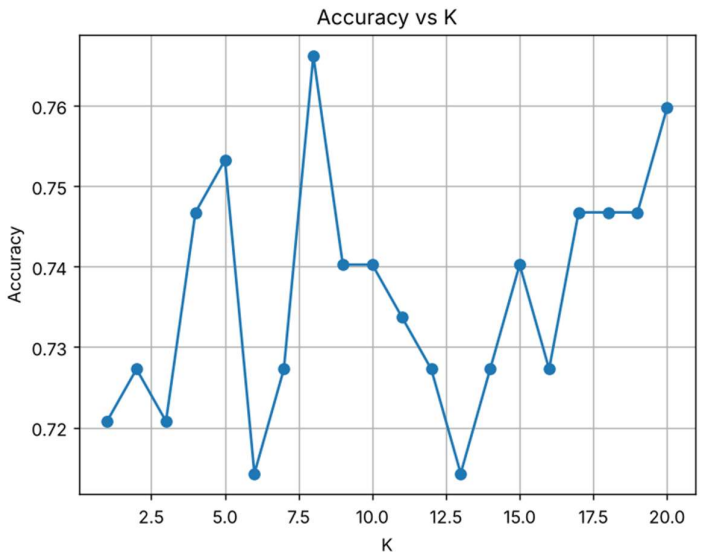


Figure: Accuracy versus K for the KNN model

Outcomes:

- Understand the working of the KNN algorithm.
- Handle invalid values in a medical dataset.
- Apply feature scaling for distance-based algorithms.
- Compute the confusion matrix, accuracy, error rate, precision and recall.
- Select a suitable value of K.

Conclusion:

In this practical the K-Nearest Neighbors algorithm was implemented on the diabetes dataset. The invalid zero values were replaced by the median, the features were standardised and the model was trained with K = 5. The performance was evaluated using the confusion matrix, accuracy, error rate, precision and recall, and the best value of K was found by comparing accuracies for different K values. The practical showed that KNN is a simple and effective classifier whose performance depends on the choice of K and on feature scaling.

Viva Questions:

1. Why is KNN called a lazy learner?
2. Why is feature scaling necessary for KNN?
3. How is the error rate related to accuracy?
4. Why is recall important in a diabetes prediction problem?
5. How do you choose the best value of K?